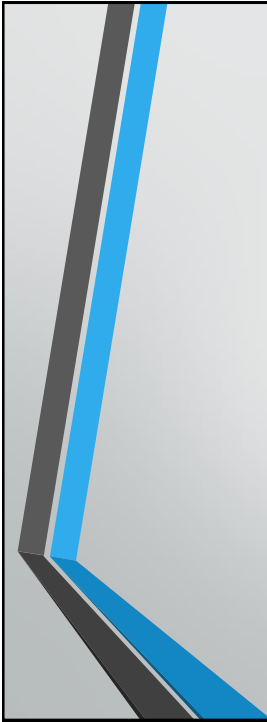




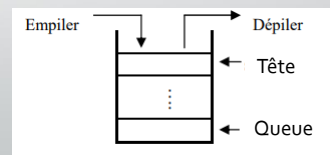
# Cours Algorithmique, Structure de Données et complexité



## Chapitre 2- Les Piles et les Files

## Les Piles

- Une Pile c'est une liste chaînée simple où l'insertion et la suppression d'éléments se fait toujours par la tête de la liste
  - **LIFO** (Last In First Out) : le dernier élément ajouté est le premier élément retiré.



3

## Insérer un élément à la pile : Empiler

Procédure Empiler (**var** Tête : Liste; X : entier)

Var P : Liste

Début

P ← allouer (taille(Element))

\*P.val ← X

\*P.suivant ← Tête

Tête ← P

Fin Empiler

4

## Récupérer un élément de la pile : Dépiler

Fonction Dépiler (**var** Tête : Liste) : entier

Var P : Liste

x : entier

Début

P ← Tête

Tête ← \* Tête.suivant

x ← \*p.val

Liberer(p)

Retourner(x)

Fin Dépiler

Si la pile n'est pas vide, la Fonction **Dépiler** renvoie la donnée qui se trouve en tête de la pile et la tête pointe sur l'élément suivant (ou contient nil si la pile est maintenant vide).

**N.B.** Avant de faire appel à la fonction Dépiler, on doit s'assurer que la pile est non vide

5

## Les Files

- Une File c'est une liste chaînée simple où l'insertion se fait à la fin de la liste (Queue) alors que la suppression se fait à partir de la tête de liste
  - **FIFO** (First In First Out) : le premier élément ajouté est le dernier élément qui va être retiré.

6

## Ajouter un élément à la file : Enfiler

Procédure Enfiler (**var** Tête : Liste; X : entier)

Var P, Queue : Liste

Début

P ← Allouer (taille(Element))

\*P.val ← X

\*P.suivant ← nil

Si (Tête = nil) alors //La file est vide

Tête ← P

Sinon //parcourir la file pour chercher sa fin (queue)

Queue ← Tête

TantQue ( \*Queue.suivant ≠ nil) faire

Queue ← \*Queue.suivant

FinTantQue

\*Queue.suivant ← P

FinSi

Fin Enfiler

7

## Retirer un élément de la file : Défiler

Fonction Défiler (**var** Tête : Liste): entier

Var P : Liste

x: entier

Début

P ← Tête

Tête ← \*Tête.suivant

x ← \*p.val

Liberer(p)

Retourner(x)

Fin Défiler

N.B. Avant de faire appel à la fonction Défiler, on s'assure que la file est non vide

8

## Les Piles et les Files

- On suppose toujours l'existence des primitives suivantes :
  - **Enfiler** (*var F : File; X: typevaleur*) : permet d'ajouter un élément de valeur X à la file F
  - **Défiler** (*var F : File*) : *fonction* qui permet d'extraire un élément à partir de la file F
  - **FileVide**( *F : File* ) : *booléen* : qui retourne vrai si la file de tête F est vide
  - **Empiler** ( *var P : Pile; X: entier* ) : qui permet d'ajouter un élément de valeur X à la pile P
  - **Dépiler** (*var P : Pile*) : *fonction* qui permet d'extraire un élément à partir de la pile P
  - **PileVide**( *P : Pile* ) : *booléen* : qui retourne vrai si la pile de tête P est vide

9

## Les Piles et les Files

- Application 1 : Ecrire l'algorithme de la procédure **AfficherPile** (*var P : Pile d'entier*) qui permet d'afficher les éléments de la Pile P

**procédure AfficherPile** (*var P : Pile d'entier*)

Var P2 : Pile d'entier

X : entier

Début

P2 ← nil

Tant Que (NON PileVide(P)) faire

    X ← dépiler (P)

    Empiler (P2, X)

    Ecrire (X )

Fin Tant Que

Tant Que (NON PileVide(P2) ) faire

    Empiler (P, Dépiler (P2))

Fin Tant Que

Fin

N.B. Faites attention à ne pas perdre le contenu de la pile initiale dans votre solution.

10

# Les Piles et les Files

Application 2 : Ecrire l'algorithme de la **Fonction RechercheP** (var P : Pile d'entier; X : entier) : booléen

**Fonction RechercheP** (var P : Pile d'entier; X : entier) : booléen

Var e : entier

trouve : booléen

P2 : Pile d'entier

Début

P2 ← nil

trouve ← faux

**TantQue** (Non PileVide (P) ) ET (trouve = faux) **Faire**

e ← Dépiler ( P)

Empiler (P2, e)

trouve ← e = X

**FinTantQue**

Tant Que (NON PileVide(P2) ) faire

Empiler (P, Dépiler (P2))

Fin Tant Que

Retourne (trouve)

**Fin RechercheP**

11

# Les Piles et les Files

Application 3: Ecrire l'algorithme de la **Procédure copier** (var F1, var F2: File d'entier)

**Procédure copierF** (var F1, var F2: File d'entier)

début

TantQue (NON FileVide(F1) ) faire

Enfiler (F2, Défiler (F1))

FinTantQue

Fin copier

12

# Les Piles et les Files

- Application 4 : Ecrire l'algorithme de la **procédure AffichFile (var F : File d'entier)**

**procédure AffichFile (var F : File d'entier)**

Var F2 : File d'entier

X : entier

Début

F2 ← Nil

TantQue (NON FileVide(F)) faire

    X ← défiler (F)

    Enfiler (F2, X)

    Ecrire ( X)

FinTantQue

TantQue (NON FileVide(F2) ) faire

    Enfiler (F, Défiler (F2))

FinTantQue

Fin

13

# Les Piles et les Files

- Application 5 : Ecrire l'algorithme de la Fonction **RechercheF (var F : file d'entier; X : entier) : booléen**

Fonction **RechercheF (var F : file d'entier; X : entier) : booléen**

Var E : entier

trouve : booléen

F2 : File d'entier

Début

F2 ← nil

trouve ← faux

TantQue (Non FileVide (F) ) Faire

    E ← Défiler (F)

    Enfiler (F2, E)

    si ( E = X) alors trouve ← vrai   FinSi

FinTantQue

CopierF (F2, F) // ou bien aussi F ← F2

Retourne (trouve)

Fin RechercheF

14